

90-DAY CAREER DEVELOPMENT PLAN

Zero → Strong Foundations → AI/Agent Skills → Portfolio → Interview Ready

DSA • Core CS • Python • AI/ML • Generative AI • AI Agents • Projects • Certifications • Career Preparation

THE 90-DAY MISSION

Build durable fundamentals first, then layer modern AI skills on top. The target is not to finish a list of courses. The target is to be able to solve problems, explain core CS concepts, build useful AI systems, show real projects, and perform in interviews.

Outcome Area	90-Day Target
DSA	100-150 quality problems, revised by pattern; timed practice included
Core CS	OOP, DBMS/SQL, OS, Networks — interview-ready fundamentals
AI/ML	Python for AI, ML fundamentals, evaluation, neural-network basics
GenAI	LLMs, prompting, structured output, embeddings, vector DB, RAG
AI Agents	Tool calling, workflows, memory, evaluation; 2 small + 1 major agent
Portfolio	2-3 meaningful projects + one polished major project
Career	GitHub, resume, LinkedIn, mock interviews, applications

How to use this PDF

- Follow the phase order, but use the daily checklist to control workload.
- Do not add extra courses just because they look interesting. Finish the required outcomes first.
- Every week ends with a measurable deliverable and a review.
- If a topic is already mastered, use the checkpoint to test it and move forward; if not, stay with it until the checkpoint is passed.

1. DESIGN PRINCIPLES

This version keeps the original plan's broad goals but makes the sequence, workload, outputs, revision system, and project progression explicit. It is designed to work even when starting from zero.

Rule	What it means
Foundation before hype	Python + DSA + Core CS come before advanced agent frameworks.
Learn → Build → Explain	Every important concept must be practiced and explained in your own words.
One source at a time	Avoid playlist/course hopping. Use one primary resource and one backup.
Projects prove skill	A project is only complete when it runs, is documented, and can be explained.
Revision is scheduled	Old DSA and CS topics return every week; no one-pass learning.
Measure outcomes	Track problems solved, concepts passed, commits, project milestones and mock scores.
Depth over certificates	2-3 relevant certifications are enough; projects must accompany them.

2. 90-DAY ROADMAP AT A GLANCE

Phase	Days	Main Focus	Exit Condition
1 — Foundation	1-30	Python, DSA basics, OOP, SQL/DBMS, AI/ML basics	Can solve basic patterns and explain fundamentals
2 — Skill Building	31-60	Intermediate DSA, OS, Networks, ML, GenAI, first agents	Can build small AI/GenAI applications and solve medium DSA
3 — Projects & Career	61-90	Advanced DSA, agents, major project, interview prep	Portfolio + interview readiness + application system

3. WEEKLY RHYTHM

Six focused days + one lighter review day is preferable to seven overloaded days. The schedule below gives a default workload; college/exam weeks can use the minimum version.

Activity	Normal Day	Minimum Day	Frequency
DSA	90-120 min	60 min	6 days/week
Core CS	45-60 min	30 min	5 days/week
AI/ML/GenAI/Agents	60-90 min	45 min	6 days/week
Project	60 min	30 min	4-5 days/week
Aptitude	30 min	20 min	3 days/week
Communication	20 min	10 min	5-6 days/week
Certification	30-45 min	—	3 days/week when active
Weekly review	45-60 min	30 min	1 day/week

Target workload: approximately 4.5-6 hours on a normal focused day, not 7+ hours every day. The minimum-day version prevents a bad week from becoming a lost week.

4. DSA MASTER PLAN

The original target of 100–150 problems is retained, but problems are organized by patterns so that solving ability—not problem count—is the outcome. [filecite]turn0file0[L97-L103]

Weeks	Topics	Target
1–2	Complexity, arrays, strings, hashing, two pointers, sliding window	18–22 problems
3–4	Linked lists, stack, queue, binary search, recursion	18–22 problems
5–6	Trees, BST, heap/priority queue, backtracking	18–22 problems
7–8	Graphs, BFS/DFS, greedy, mixed patterns	18–22 problems
9–10	DP basics, graph practice, binary-search patterns, mixed medium	18–22 problems
11–12	Timed sets, weak topics, mixed interview practice	15–20 problems
13	Mock contests + final revision	8–12 problems

DSA learning loop

Step	Action
1. Learn	Understand the pattern and write the idea without copying code.
2. Implement	Code from scratch in Python.
3. Solve	Attempt before looking at the solution; use a time limit.
4. Explain	State approach, invariant/logic, complexity and edge cases.
5. Log	Record the pattern, mistake and why the solution works.
6. Revisit	Retry after 3–7 days and again before the phase checkpoint.

DSA checkpoint: by Day 30, comfortably solve basic array/string/hash/two-pointer/sliding-window/binary-search questions; by Day 60, handle common trees/graphs/greedy patterns; by Day 90, perform timed mixed sets.

Python fundamentals required for DSA

- Variables, conditions, loops, functions and recursion
- Lists, tuples, sets, dictionaries, strings and slicing
- Sorting, custom keys, lambda, enumerate, zip
- Stack/queue patterns, heapq, collections.deque
- Input/output, debugging and basic time/space complexity

5. CORE COMPUTER SCIENCE

The plan covers OOP, DBMS/SQL, OS and Networks. The improvement here is to attach interview outputs to each topic rather than treating them as reading-only subjects.

Topic	Learn	Must be able to answer/build
OOP	Classes, objects, inheritance, polymorphism, abstraction, composition	Explain with a small Python example; compare composition vs inheritance
SQL	SELECT, filtering, joins, grouping, subqueries, constraints, indexes	Write queries from a schema without notes
DBMS	Keys, normalization, transactions, ACID, isolation, indexing	Explain why indexes and transactions matter
OS	Processes, threads, scheduling, memory, synchronization, deadlock	Explain process vs thread and common scheduling/deadlock concepts
Networks	OSI/TCP-IP, HTTP, DNS, TCP/UDP, basic APIs	Trace what happens when a browser requests a website

Core CS weekly method

- Day 1-2: learn concepts and make a one-page note.
- Day 3: solve 5-10 interview-style questions.
- Day 4: explain the topic aloud without notes.
- Day 5: connect the topic to a real project or system.
- Weekly review: answer a 10-question closed-book quiz.

High-value interview questions to master

- Why is a hash table average $O(1)$? When does it degrade?
- Process vs thread; context switching; synchronization.
- Why do databases need indexes? What trade-off do they create?
- ACID and transaction isolation in practical terms.
- TCP vs UDP; HTTP request lifecycle; DNS resolution.
- OOP principles and when composition is preferable.

6. AI / ML FOUNDATION

The source plan starts AI with Python, NumPy/Pandas, AI vs ML vs DL vs GenAI, then regression/classification, datasets, train/test and evaluation, followed by feature engineering, overfitting, regularization and neural-network basics. [filecite turn0file0 L17-L32](#)

Stage	Concepts	Practical output
AI foundations	AI vs ML vs DL vs GenAI; Python for AI	Explain the hierarchy with examples
Data	NumPy/Pandas overview, cleaning, train/test split	Clean and inspect a small dataset
Supervised ML	Regression, classification	Train at least one model of each type
Evaluation	Appropriate metrics, validation, error analysis	Compare models and explain metric choice
Generalization	Overfitting, underfitting, regularization	Demonstrate a model that overfits and improve it
Neural networks	Layers, activation, loss, optimization basics	Build a small neural-network experiment

Mini-project 1: ML/AI project

- Choose a small real dataset with a clear question.
- Document data preparation, baseline, model choice, evaluation and limitations.
- Keep the scope small enough to finish within 1–2 weeks.
- Publish a clean README and a short demo/result section.

Checkpoint before moving to GenAI

- Can explain train vs test and why leakage is dangerous.
- Can choose a reasonable metric for a basic classification/regression problem.
- Can describe overfitting and at least two ways to reduce it.
- Can read a simple Python ML notebook without feeling lost.

7. GENERATIVE AI + RAG

Weeks 7-8 in the source introduce LLMs, prompting, embeddings, vector databases and RAG; this section turns that into a concrete build sequence. [filecite turn0file0 L33-L38](#)

Order	Learn	Build / prove it
1	LLM basics and API usage	Call an LLM API and handle structured output
2	Prompting	Create prompts with constraints, examples and validation
3	Structured output	Return predictable JSON and validate it
4	Embeddings	Embed text and compare semantic similarity
5	Vector DB	Store/retrieve chunks by similarity
6	RAG	Build retrieval → context → answer pipeline
7	RAG quality	Test retrieval failures, hallucinations and missing context

Project 2: GenAI / RAG application

- Use a bounded document set rather than trying to index the whole internet.
- Show ingestion, chunking, embeddings, retrieval and answer generation.
- Add citations/source references inside the app where appropriate.
- Create 15-20 test questions, including questions whose answers are not in the documents.
- Document limitations and failure cases.

RAG is not 'just a chatbot'. The project must demonstrate retrieval quality, grounding and failure handling.

8. AI AGENT LEARNING PATH

The source specifies prompts, LLM APIs, structured outputs, tool/function calling, embeddings/RAG, memory, workflows, planning, error handling, agent frameworks, evaluation, security and reliability.

filecite turn0file0 L70-L78

Level	Capability	Evidence
L1	LLM + structured output	Reliable JSON task
L2	Tool/function calling	Agent invokes 2-3 deterministic tools
L3	Workflow	Multi-step stateful execution
L4	RAG agent	Agent retrieves knowledge before acting
L5	Memory	Useful short/long-term state with clear boundaries
L6	Reliability	Retries, validation, timeouts, fallbacks, logging
L7	Evaluation	Test set + success criteria + failure analysis
L8	Security	Tool permissions, input validation, secrets handling

Agent Project A — Small practical agent

Example scope: a research/planning assistant that can call a search-like tool, calculator, note store or task tool. Keep the tool set small and deterministic.

Agent Project B — RAG + tools

Combine retrieval with at least one action tool. Add validation and a visible execution trace so the user can understand what happened.

Major Agent Project

- Must solve a real multi-step problem rather than only chat.
- Must have tools, state/workflow, error handling and an evaluation set.
- Must have architecture documentation and a 2-3 minute demo.
- Must clearly state what the agent cannot do and where human approval is required.

9. PORTFOLIO SYSTEM

Project	Purpose	Definition of Done
Project 1 — ML/AI	Demonstrate data + model fundamentals	Working repo + README + evaluation + limitations
Project 2 — GenAI/RAG	Demonstrate LLM + retrieval	Working app + retrieval pipeline + test set + demo
Project 3 — AI Agent	Demonstrate tools + multi-step execution	Working agent + tool calls + error handling + demo
Major — SATS	Turn the existing major project into a structured system	Architectural system agent functionality + docs + demo + measurable contribution

Every repository must contain

- Problem statement and target user
- Features and architecture diagram
- Technology stack and setup instructions
- Screenshots or demo link
- Known limitations and future work
- Your exact contribution
- A short 'How it works' section that you can explain in an interview

Career preparation

When	Action
Day 1	Create/clean GitHub profile and learning tracker
Weekly	Push meaningful commits and update project notes
Day 30	Create first resume draft
Day 45-60	Begin project-focused resume bullets with measurable results
Day 75	Mock technical interview + resume review
Day 85	Finalize resume, LinkedIn and project links
Day 90	Start/continue internship and hackathon applications

10. CERTIFICATIONS + AI UPDATE SYSTEM

The source recommends 2-3 relevant certifications, prioritizing Generative AI, AI fundamentals, Machine Learning and/or Cloud AI, with a mini-project tied to each certification. [filecite turn0file0 L87-L93](#)

Certification rule	Implementation
Choose 2-3	Do not collect certificates for appearance.
Match the roadmap	Certification should reinforce a topic already being learned.
Build after learning	Create a mini-project or practical artifact from the material.
Record proof	Keep certificate + project + short reflection in your career tracker.

Weekly AI update tracker

Date	Tool / Model / Topic	What changed?	What I learned	Tried?

Rule: spend at most 30-60 minutes per week on AI news unless a development directly affects a project. The purpose is awareness, not endless scrolling.

Learning tracker fields

- Date • Topic • Resource • Time spent • Problems solved • Project progress • Mistake learned • Confidence (1-5) • Next action

11. WEEK-BY-WEEK EXECUTION PLAN

Week	DSA	Core CS	AI	Project / Career	Checkpoint
1	Complexity + arrays	OOP basics	Python for AI	GitHub + tracker	20-question basics test
2	Strings + hashing + two pointers	SQL basics	NumPy/Pandas	Small coding exercises	Solve 5 mixed DSA timed
3	Sliding window + linked list	DBMS basics	Regression	ML project starts	ML train/test explained
4	Stack + queue + binary search	Optimizations/normalization/transactions	Classification + evaluation	ML project finished	Day-30 review
5	Trees + BST	OS processes/threads	Feature engineering	README + GitHub polish	Tree pattern test
6	Heap + backtracking	OS scheduling/memory/deadlocks	Overfitting + NN basics	Project explanation practice	Core CS quiz
7	Graphs BFS/DFS	Networks basics	LLMs + prompting	GenAI project starts	API + structured output
8	Greedy + mixed	HTTP/DNS/TCP/UDP	Embeddings + RAG	RAG project	RAG test set
9	DP basics	Revision	Tool calling	Agent A	2 tools working
10	DP + graph practice	Revision	Memory/workflows/evaluation	Agent B	Agent evaluation
11	Timed mixed DSA	Interview revision	Agent reliability/security	Major project	Mock interview 1
12	Weak-topic repair	Full Core CS review	Deployment/API basics	Major project + docs	Resume + mock 2
13	Mocks + final revision	Final technical review	Explain architecture	Applications + final report	90-day assessment

Important: the week table is a navigation map, not permission to rush. If a checkpoint fails, repair the prerequisite before adding more advanced material.

12. DAILY TEMPLATE + CHECKPOINTS

Default focused day

Block	Work
Block 1 — DSA	90–120 min: learn/solve/revise one pattern
Block 2 — Core CS	45–60 min: concept + interview questions
Block 3 — AI	60–90 min: learn + implement
Block 4 — Project	60 min: build/document/test
Block 5 — Career	20–30 min on selected days: aptitude/communication/certification

Weekly review questions

- What did I actually build or solve this week?
- Which DSA patterns can I solve without hints?
- Which Core CS topic can I explain without notes?
- What AI concept can I demonstrate in code?
- What failed in my project and what did I change?
- What should be removed from next week's plan to protect the priorities?

Phase checkpoints

Checkpoint	Pass criteria
Day 30	Foundation DSA patterns + SQL/OOP + ML basics demonstrated; Project 1 complete or near-complete
Day 60	Intermediate DSA + OS/Networks + GenAI/RAG + first agent demonstrated
Day 75	Major project architecture and agent workflow working
Day 90	Portfolio polished, mock interviews completed, resume/LinkedIn ready, application process started

Recovery rule

If you miss a day, do not double the workload the next day. Resume from the current priority and use the weekly review to recover. Consistency beats heroic catch-up.

13. 90-DAY SUCCESS CHECKLIST

Target	Done?
100–150 quality DSA problems solved and revised	☐
Python fundamentals strong enough for DSA and AI work	☐
OOP, SQL/DBMS, OS and Networks revised for interviews	☐
AI/ML fundamentals understood and demonstrated	☐
LLMs, structured outputs, embeddings and RAG understood	☐
At least 2 small AI agents built	☐
One major agent-based system documented	☐
2–3 meaningful AI/GenAI projects completed	☐
2–3 relevant certifications completed or intentionally selected	☐
GitHub repositories polished with README + architecture + demo	☐
Resume and LinkedIn updated with measurable project evidence	☐
Aptitude + communication practice completed	☐
At least 2 mock interviews / timed technical sessions completed	☐
Internship/hackathon applications started	☐
Final 90-day report completed	☐

FINAL DEFINITION OF SUCCESS

At the end of 90 days, success means you can solve unfamiliar coding problems with a repeatable process, explain the major Core CS concepts, build and debug AI/GenAI/agent systems, show credible projects on GitHub, and communicate your work clearly in an interview.

The plan is intentionally zero-to-strong: do not assume prior knowledge. The sequence starts with foundations, increases difficulty only after checkpoints, and converts learning into visible evidence.

One sentence to remember

Learn deeply → solve deliberately → build publicly → revise repeatedly → explain clearly → apply consistently.